

Sonae Page-Speed Optimisation Report

Evidence-backed summary of what was optimised, the measured gains, rejected experiments, exclusions, and verification.

Production-like audit completed 25 August 2026. Scope: every eligible non-Posture-Studio page in the Sonae repository.

Executive Summary

The optimisation loop audited **166 of 166 eligible routes** and retained only changes that improved an identical production-like measurement. The strongest gains came from delaying Recharts, Markdown, message renderers, complete screens, dialogs, and interaction-only helpers until the data or user action that needs them. The final /admin/agents/skills change removed **8,863 Brotli bytes**, one script, and one unnecessary page response while keeping the same destination and permissions.

No commit, push, deployment, merge, publication, or pull request was performed.

Eligible routes audited

166

Every eligible non-Posture page received a recorded audit result.

Source: Sonae Page Speed Optimiser evidence ledger

Table: route_gains

Audit coverage

100%

Coverage uses 181 total page routes minus 14 path exclusions and the mixed public home route.

Source: Sonae Page Speed Optimiser evidence ledger

Table: route_gains

Excluded routes

15

Fourteen Posture or movement paths plus the mixed public home page remained outside scope.

Source: Sonae Page Speed Optimiser evidence ledger

Table: route_gains

Tests passed

6,039

The final retained root passed the complete repository verification suite.

Test files **684**

Source: Sonae Page Speed Optimiser evidence ledger

Table: route_gains

Final-route Brotli saved

8.86K

The last legacy Agent Skills optimisation also removed one unnecessary page response.

Source: Sonae Page Speed Optimiser evidence ledger

Table: route_gains

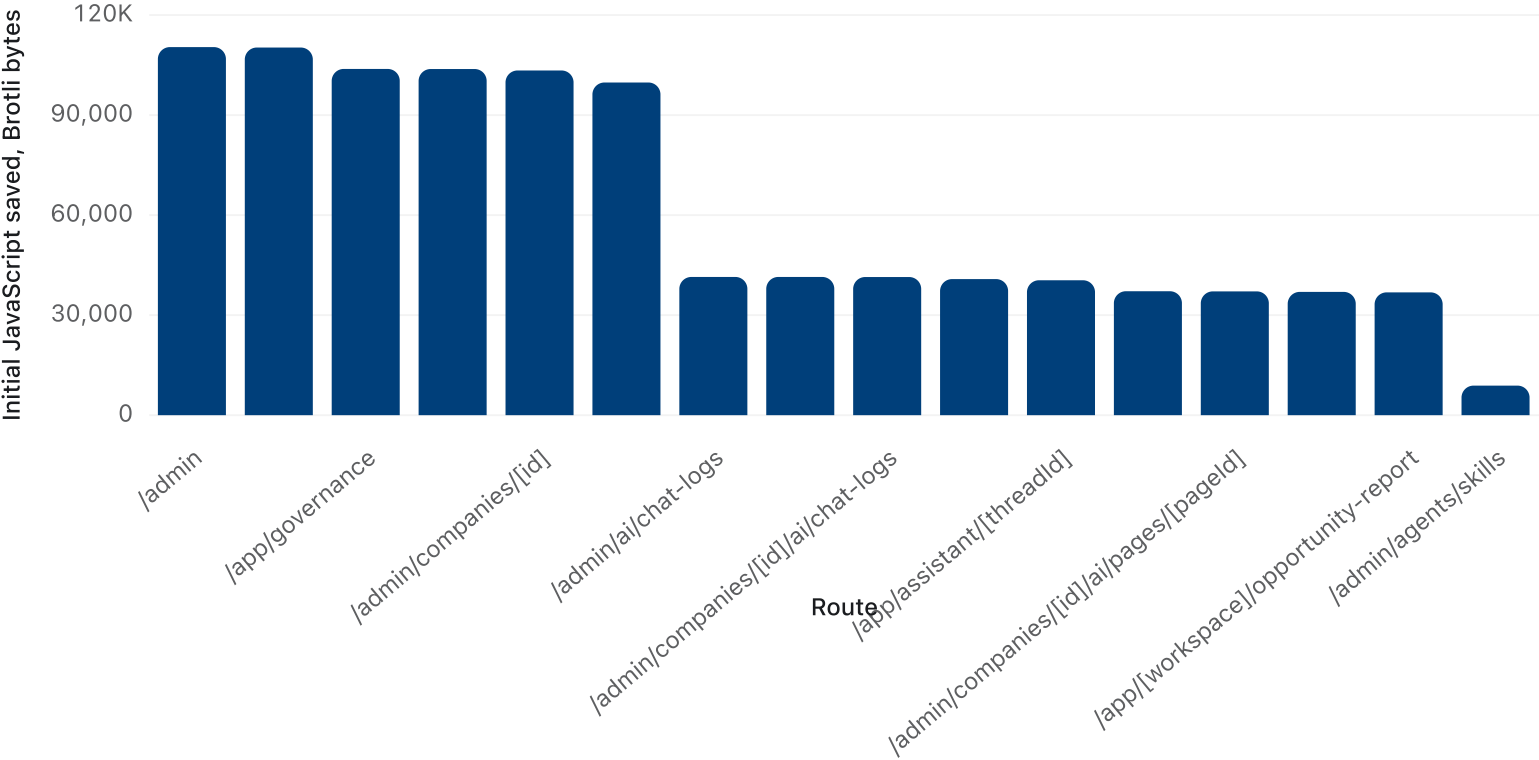
Key Finding

Initial-load cost was concentrated in code that could not be used on first paint. Deferring charting and rich-text/message rendering produced the largest route-level gains, often removing **70%–94%** of route-exclusive Brotli JavaScript on the highlighted pages.

Largest measured route-level Brotli savings

Source: Sonae Page Speed Optimiser evidence ledger

Table: route_gains



Notable retained route gains

Source: Sonae Page Speed Optimiser evidence ledger

Table: route_gains

Rank	Route	Pattern	Before	After	Saved	Reduction %	What changed
1	/admin	Deferred charts	117.85K	7.46K	+110.39K	93.7	Load Recharts only after platform data resolves
2	/app/reports	Deferred charts	158.36K	48.1K	+110.26K	69.6	Load report charts only after the report query resolves
3	/app/governance	Deferred charts	114.04K	10.2K	+103.84K	91.1	Defer the governance runs chart until data exists
4	/admin/governance	Deferred charts	113.38K	9.56K	+103.82K	91.6	Defer the shared governance chart until data exists
5	/admin/companies/[id]	Deferred charts	110.08K	6.7K	+103.38K	93.9	Load company charts after engagement data resolves
6	/admin/companies/[id]/ai/usage	Deferred charts	106.45K	6.68K	+99.77K	93.7	Load usage charts after company metrics resolve
7	/admin/ai/chat-logs	Deferred Markdown/messages	52.9K	11.45K	+41.46K	78.4	Load messages and Markdown only after thread selection
8	/admin/ai/usage/chat-logs	Deferred Markdown/messages	52.9K	11.45K	+41.46K	78.4	Shared chat-log optimisation on the alias route
9	/admin/companies/[id]/ai/chat-logs	Deferred Markdown/messages	51.62K	10.19K	+41.43K	80.3	Load messages and Markdown after company thread selection

Rank	Route	Pattern	Before	After	Saved	Reduction %	What changed
10	/app/agentic-testing	Deferred Markdown/messages	48.73K	7.91K	+40.81K	83.8	Load ChatMessage and Markdown after a sandbox thread exists
11	/app/assistant/[threadId]	Deferred Markdown/messages	55.07K	14.61K	+40.46K	73.5	Load message rendering after the thread query resolves
12	/admin/ai/knowledge/[pageId]	Deferred Markdown	46.62K	9.44K	+37.17K	79.7	Load Wiki prose only after detail data resolves
13	/admin/companies/[id]/ai/pages/[pageId]	Deferred Markdown	46.52K	9.4K	+37.12K	79.8	Shared Wiki detail optimisation for company pages
14	/w/[widgetId]	Deferred Markdown	49.4K	12.43K	+36.97K	74.8	Load Markdown only after the widget receives an answer
15	/app/[workspace]/opportunity-report	Deferred Markdown	48.45K	11.62K	+36.83K	76	Load report Markdown only for a completed summary

16 results · Showing first 15

What Was Optimised

The work preserved visible UI, wording, permissions, tenant isolation, queries, mutations, navigation, and user flow. It changed *when code loads*: after successful queries, thread selection, identity checks, open actions, or voice/upload gestures. Redirect-only routes were made server-side where safe, and the final legacy Skills path now redirects authenticated users before its obsolete client bundle is requested.

What was optimised

Source: Sonae Page Speed Optimiser evidence ledger

Table: route_gains

Order	Optimisation pattern	Where used	Measured gain	Behaviour-preserving outcome
1	Defer charts until data exists	Reports, platform/admin dashboards, governance, company analytics	99.8–110.4 KB Brotli on the largest routes	Removed Recharts from initial entries while preserving chart output and loading states
2	Defer Markdown and message renderers	Assistant, chat logs, Wiki detail, widgets, opportunity reports	36.8–41.5 KB Brotli on highlighted routes	Renderer code loads only after a thread, answer, or document is available
3	Gate complete screens behind existing queries or identity	Security, invitations, features, rule/eval screens	Up to 5.1 KB Brotli in later slices	Kept the same spinner or access gate; moved the loaded screen out of the initial path
4	Defer closed dialogs and action-only tools	Retention, plans, evals, API keys, rules, schedules	Tens of bytes to 1.7 KB Brotli per route	Dialogs preload on the existing action and mount only when opened
5	Defer interaction-only helpers	Kiosk voice, previews, uploads, validation, map layout	About 0.5–1.7 KB Brotli on representative routes	PCM, validation, upload, and layout helpers move behind the action that needs them
6	Remove redundant legacy navigation	Legacy Agent Skills and redirect-only aliases	8.9 KB Brotli plus one page response on Agent Skills	Authenticated users reach the existing destination before the unreachable legacy page loads

Recommendations

1. Review and batch the retained local changes before any commit or push; the automation deliberately published nothing.
2. Add route-level bundle budgets for the largest wins so Recharts and Markdown do not drift back into initial entries.
3. Use real-user LCP, INP, and navigation timing after release to translate byte savings into user-perceived milliseconds.

4. Keep the same retain-or-revert rule: identical production build, compressed measurement, focused tests, full gates, and no visible behaviour change.

Further Questions

- Should the mixed public home page ever enter the audit? That requires an explicit product decision to decouple its direct Posture Studio panel first.
- Which routes deserve real-user monitoring first? The chart-heavy admin/dashboard pages and message-heavy assistant surfaces have the largest measured bundle reductions.
- Should this report be published as a shareable internal Site after review?

Caveats

- These gains are initial JavaScript bundle bytes, not direct milliseconds. Network, device, cache, server response, and rendering conditions determine experienced load time.
- Route-level savings are **not summed into one portfolio total** because shared chunks appear on multiple routes and would be double-counted.
- /admin/ai/costs saved **140,726 gzip bytes** and /app/settings saved **119,142 gzip bytes**, but they are not ranked in the Brotli chart because their early ledger entries did not record exact Brotli savings.
- Fourteen Posture/movement paths and the mixed public / route were excluded. Mandatory repository verification could execute existing excluded tests, but no excluded code was targeted or changed.
- Rejected experiments were restored when compressed bytes regressed, even if raw bytes improved.